ROUND 19 — CONSTITUTION V6: INSTITUTIONAL ADAPTATION

Institutional Adaptation

V3 observed. V4 judged. V5 disappeared. V6 adapts. Memory says "we've seen this." Evolution says "we no longer make this mistake."

Reviewer	Independent (Super Z, Z.ai) — acting as Principal Engineer specifying V6
Baseline	Commit a044a4a. V5 Phase 1 complete (organs hidden, executive function, attention allocation — quality fixed). 63 backend modules, 27 frontend files. V5 Specs #4-#8 NOT built.
V6 shift	V5 = invisible intelligence that assists. V6 = adaptive intelligence that reshapes. Maestro stops reporting problems and starts quietly restructuring work to prevent them. The moat shifts from memory to evolution.
Constitutional laws	Law 1: "Every interaction must permanently improve the organization." Law 2: "The organization should become more intelligent even when nobody opens Maestro." Both are mandatory.
Deliverable	7 specifications across 3 phases: (1) finish V5 + add adaptive nudges, (2) institutional evolution tracking + background adaptation, (3) organizational DNA + trajectory intervention. Each has API + acceptance test + build order.

WHY V6 EXISTS

V5 proved that intelligence can become invisible. The organs are hidden. The executive function acts. The attention allocates. The UI is simpler. That is a significant achievement — but it is still "memory that assists." Maestro remembers what happened, judges what to do, and helps you do it. The user still opens Maestro (or Maestro visits them) and receives assistance.

V6 is different. V6 says: stop assisting. Start adapting. Maestro should not report that "Legal is the bottleneck" — it should quietly route OAuth approvals through Alice first, because historical evidence suggests an 18% reduction in review time. Nobody asked. Nobody configured it. The organization simply improves. The user never realizes something was prevented. That is an operating system, not an assistant.

The V6 constitutional laws (both mandatory):

Law 1: "Every interaction must permanently improve the organization." Not the model. Not the UI. Not the database. The organization. If nothing permanently improves, Maestro failed.

Law 2: "The organization should become more intelligent even when nobody opens Maestro." This forces every capability toward ambient, invisible, anticipatory, cumulative — not dashboards, reports, pages, clicks.

The progression is clear: V3 Observe → V4 Judge → V5 Disappear → V6 Adapt → V7 Evolve. V6 is the first constitution where Maestro changes reality, not merely understanding. The moat shifts from "we've seen this" (memory) to "we no longer make this mistake" (evolution). That is the asset that commands strategic acquisition interest — not software, but years of institutional evolution encoded in the system.

Same discipline as V3/V4/V5. Every spec has: the principle, the current codebase gap, exact files, API contract, acceptance test, effort, build phase. The V5 litmus test ("does this make Maestro feel simpler?") is retained AND augmented with the V6 litmus test: "does this permanently improve the organization?" Both must pass.

The 7 V6 Specifications

V6 has 7 specs in 3 phases. Phase 0 finishes the V5 backlog (Specs #4-#8 from V5, which were not built). Phase 1 adds adaptive nudges — Maestro quietly restructures work. Phase 2 adds institutional evolution tracking and background adaptation. Phase 3 adds organizational DNA and trajectory intervention. Total ~24 days including V5 backlog.

#	Specification	Phase	Effort	What It Does
0a	V5 Spec #4: Forgetting Engine	0	1.5 days	Archive zero-predictive-value events. V5 backlog.
0b	V5 Spec #5: Imagination (Counterfactual)	0	2 days	"What if Legal disappeared?" V5 backlog.
0c	V5 Spec #6: Causal Cognition	0	2 days	Correlation → causation. V5 backlog.
0d	V5 Spec #7: Temporal Trajectories	0	1.5 days	Org-wide trajectory memory. V5 backlog.
0e	V5 Spec #8: Institutional Recall	0	2 days	"When have we been here before?" V5 backlog.
1	Adaptive Nudge Engine	1	3 days	Maestro quietly restructures work (route approvals, change cadence)
2	Institutional Evolution Tracker	1	2 days	"We no longer make this mistake" — tracks permanent improvement
3	Background Adaptation Loop	2	2 days	Organization improves even when nobody opens Maestro
4	Trajectory Intervention	2	2.5 days	Weak signal → quiet intervention → failure prevented
5	Organizational DNA	3	3 days	Inferred decision style that evolves and drives recommendations
6	Evolution Narrative Engine	3	2 days	"Your organization has changed" — the autobiography
TOTAL	**7 V6 specs + 5 V5 backlog**	**4 phases**	**~24 days**	**V6: from invisible intelligence to institutional adaptation**

Build order: Phase 0 (V5 backlog, ~9 days) → Phase 1 (Adaptive Nudges + Evolution Tracker, ~5 days) → Phase 2 (Background Adaptation + Trajectory Intervention, ~4.5 days) → Phase 3 (Org DNA + Evolution Narrative, ~5 days). Total ~24 days. Phase 0 is mandatory — V6 specs build on V5 capabilities (causal cognition, temporal trajectories, institutional recall) that do not exist yet.

Phase 0 — V5 Backlog (Must Complete Before V6)

V5 Specs #4-#8 were specified in round 17 but not built. V6 specs build on them: Adaptive Nudges require causal cognition (#6) to know which interventions work. Trajectory Intervention requires temporal trajectories (#7) to detect trajectory changes. Evolution Tracking requires institutional recall (#8) to compare "before" and "after." Phase 0 finishes V5 before starting V6.

Specs #0a-#0e: V5 Backlog (Forgetting, Imagination, Causal, Temporal, Recall)

These 5 specs are unchanged from the V5 specification (round 17). The coder should build them in the order specified: #0d (Temporal, 1.5 days) → #0c (Causal, 2 days) → #0a (Forgetting, 1.5 days) → #0b (Imagination, 2 days) → #0e (Recall, 2 days). Total ~9 days. The acceptance tests are identical to the V5 spec — see round 17 report for details. Do NOT build V6 specs until Phase 0 is complete. V6 builds on V5 capabilities that do not exist yet.

> **PHASE 0 IS MANDATORY. DO NOT SKIP TO V6.**
>
> The "built but not applied" pattern was broken in round 16. The "silently skipped" pattern was caught in round 13. DO NOT introduce a new variant: "skipped Phase 0 and built V6 on a missing foundation." V6 Spec #1 (Adaptive Nudges) requires causal cognition to know which interventions work. V6 Spec #4 (Trajectory Intervention) requires temporal trajectories to detect trajectory changes. V6 Spec #2 (Evolution Tracker) requires institutional recall to compare before/after. Build Phase 0 first. Then build V6.

Phase 1 — Adaptive Nudges + Evolution Tracking

Phase 1 is the V6 foundation. V5 made Maestro invisible; V6 makes it active. The Adaptive Nudge Engine is the organ that separates an assistant from an operating system: Maestro stops reporting problems and starts quietly restructuring work to prevent them. The Evolution Tracker measures whether the restructuring actually improved the organization.

SPEC #1 Adaptive Nudge Engine — Maestro quietly restructures work

V6 Principle

V6 Law 1: "Every interaction must permanently improve the organization." Instead of reporting "Legal is the bottleneck," Maestro quietly suggests: "Beginning next Monday, route OAuth approvals through Alice first. Historical evidence suggests an 18% reduction in review time." Nobody asked. Nobody configured it. The organization simply improves.

Current codebase gap

No nudge/adaptation module exists. The closest is executive_function.py (207 lines) which produces a PLAN but does not SUGGEST restructuring. The attention.py engine identifies attention thieves but does not propose routing changes. The gap: Maestro identifies problems and plans solutions, but does not proactively suggest work restructuring based on what has worked before. It reports; it does not adapt.

Files to create/modify

CREATE backend/maestro_oem/adaptive_nudge.py — the AdaptiveNudgeEngine. Composes: (1) pattern detection (from pattern.py — what recurring problems exist), (2) causal cognition (from causal.py — what interventions have worked), (3) executive function (from executive_function.py — how to implement the nudge), (4) identity (from identity.py — does the nudge align with who the organization is?). Produces nudges: {problem, intervention, evidence, expected_improvement, implementation, confidence}. CREATE GET /api/oem/nudges endpoint. MODIFY static/js/today.js — replace the static "one decision" item with a nudge card when a nudge is available: "Maestro suggests: route OAuth approvals through Alice. Evidence: 3 similar interventions reduced review time 18%." with Accept/Dismiss buttons. MODIFY static/js/cognition.js — a "What Maestro is changing" section showing active nudges.

API contract

GET /api/oem/nudges → returns JSON: { "nudges": [{ "problem": "Legal review is the bottleneck for OAuth approvals", "intervention": "Route OAuth approvals through Alice (carlos.r@acme.com) first, before full Legal review", "evidence": "3 similar routing changes produced an 18% reduction in review time", "expected_improvement": "18% faster approvals, 2-day reduction in cycle time", "implementation": "Update the approval workflow to cc Alice on OAuth-related PRs. Maestro can draft the workflow change.", "confidence": 0.78, "status": "suggested", "accepted_at": null, "causal_chain_id": "causal-xxx" }], "active_nudges": 0, "suggested_nudges": 1, "summary": "Maestro has 1 suggestion for improving how your organization works." }. Each nudge must have problem, intervention, evidence, expected_improvement, implementation, confidence (0-1). The evidence must reference causal data (from causal.py) — not just correlation.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/nudges. Response must have at least 1 nudge with all 6 fields non-empty. 2) Auditor verifies the evidence references causal data (not just "this pattern exists" — must say "this intervention worked N times"). 3) Auditor verifies the nudge is actionable (not just "address the bottleneck" — must be a specific restructuring like "route through Alice"). 4) Auditor opens TODAY and verifies a nudge card appears with Accept/Dismiss buttons. 5) Auditor verifies the nudge does NOT appear if no causal evidence exists (honest degradation — "Maestro has no restructuring suggestions yet. It needs more intervention data."). 6) Auditor verifies Accept sets status to "active" and records the acceptance.

Effort

3 days (1.5 days nudge-generation engine + 0.5 day API + 1 day frontend TODAY + Cognition integration)

Build phase

Phase 1 (first — after Phase 0 complete. Requires causal.py for evidence.)

SPEC #2 Institutional Evolution Tracker — "We no longer make this mistake"

V6 Principle

V6: "Memory says 'we've seen this.' Evolution says 'we no longer make this mistake.' Those are completely different products." The Evolution Tracker measures whether the organization has PERMANENTLY improved. It compares the frequency of past mistakes to current frequency and tracks whether interventions (nudges, executive function plans) actually changed behavior.

Current codebase gap

evolution_report.py (178 lines) produces a quarterly report with deltas (decision_making, knowledge_discipline, etc.). But it does not track SPECIFIC mistakes that have been eliminated. It says "decision_making improved 11%" but not "the OAuth approval bottleneck that recurred 5 times in Q3 has not recurred in 90 days." The gap: no mechanism to track whether a specific organizational failure mode has been permanently resolved.

Files to create/modify

CREATE backend/maestro_oem/evolution_tracker.py — the EvolutionTracker. Maintains a registry of organizational failure modes (from contradictions, patterns, invalidated assumptions). For each failure mode, tracks: first_observed, last_observed, frequency_history, current_status (active/resolving/resolved/eliminated). A failure mode is "eliminated" when it has not recurred for >= 90 days after an intervention. CREATE GET /api/oem/evolution-tracker endpoint. MODIFY static/js/learn.js — add a "Mistakes your organization no longer makes" section showing eliminated failure modes with their resolution story.

API contract

GET /api/oem/evolution-tracker → returns JSON: { "failure_modes": [{ "name": "OAuth approval bottleneck", "first_observed": "2024-09-15...", "last_observed": "2024-11-20...", "frequency_history": [{"period": "2024-09", "count": 3}, {"period": "2024-10", "count": 2}, {"period": "2024-11", "count": 1}, {"period": "2024-12", "count": 0}, {"period": "2025-01", "count": 0}], "current_status": "eliminated", "eliminated_at": "2025-02-20...", "intervention": "Routed OAuth approvals through Alice first. Causal evidence: 3 similar interventions reduced review time 18%.", "narrative": "The OAuth approval bottleneck recurred 6 times between September and November 2024. After routing approvals through Alice in December, it has not recurred in 90 days. This failure mode is eliminated." }], "active_count": 2, "resolving_count": 1, "eliminated_count": 1, "summary": "Your organization has eliminated 1 failure mode and is resolving 1 more. 2 failure modes are still active." }. Must have at least 1 failure mode with frequency_history and current_status.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/evolution-tracker. Response must have failure_modes array (1+) with frequency_history and current_status. 2) Auditor verifies each failure mode references real model data (contradictions, patterns, or invalidated assumptions — not hardcoded). 3) Auditor verifies "eliminated" status requires >= 90 days without recurrence (or honestly says "no failure modes have been eliminated yet — the pilot is too young"). 4) Auditor opens LEARN and verifies a "Mistakes your organization no longer makes" section appears. 5) Auditor verifies the narrative is a story (not a metric dump). 6) Auditor verifies the V6 litmus test: "Does this permanently improve the organization?" — YES, it tracks permanent improvement.

Effort

2 days (1.5 days tracking engine + 0.5 day API + LEARN integration)

Build phase

Phase 1 (second — after Adaptive Nudges. Nudges create interventions; the tracker measures whether they worked.)

Phase 2 — Background Adaptation + Trajectory Intervention

Phase 2 makes Maestro active even when nobody is looking. The Background Adaptation Loop runs continuously, detecting weak signals and proposing quiet interventions. The Trajectory Intervention organ detects when an organizational trajectory is heading toward failure and intervenes before the failure occurs. The user never realizes something was prevented.

SPEC #3 — Background Adaptation Loop — organization improves even when nobody opens Maestro

V6 Principle

V6 Law 2: "The organization should become more intelligent even when nobody opens Maestro." This forces every capability toward ambient, invisible, anticipatory, cumulative. The Background Adaptation Loop runs on every signal ingest (not on user request), checks for improvement opportunities, and queues nudges for the next time the user interacts.

Current codebase gap

The existing _weekly_snapshot_loop (main.py line 96-122) runs weekly to capture metrics. The _trigger_learning_resolution_locked (oem_state.py line 255) runs on signal ingest to resolve predictions. But neither PROACTIVELY generates nudges or checks for improvement opportunities on ingest. The gap: Maestro only generates nudges when the user opens it. V6 Law 2 demands it generate nudges in the background, continuously.

Files to create/modify

CREATE backend/maestro_oem/background_adaptation.py — the BackgroundAdaptationLoop. On every signal ingest (hooked into oem_state.py live_ingest), the loop: (1) checks if the new signal creates a new pattern that warrants a nudge, (2) checks if any active nudge should be escalated (the problem is getting worse), (3) checks if any resolved failure mode is recurring (regression detection), (4) queues any generated nudges for the next user interaction. CREATE GET /api/oem/adaptation/status endpoint (shows what the background loop has detected). MODIFY backend/maestro_api/oem_state.py live_ingest() — call the background adaptation loop after each signal batch. MODIFY static/js/today.js — if background-detected nudges exist, show them with a "Maestro noticed this while you were away" label.

API contract

GET /api/oem/adaptation/status → returns JSON: { "background_nudges": [{"problem": "...", "intervention": "...", "detected_at": "...", "signal_that_triggered": "..."}], "regression_alerts": [{"failure_mode": "...", "was_eliminated": true, "recurrence_detected": true, "last_seen": "..."}], "escalation_alerts": [{"nudge_id": "...", "problem_worsening": true, "current_severity": "high"}], "last_run": "...", "signals_processed_since_last_run": 12, "summary": "Maestro noticed 1 improvement opportunity and 1 regression while you were away." }. Must have at least 1 item across the 3 arrays (or honestly say "no background activity — the organization is stable").

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/adaptation/status. Response must have background_nudges, regression_alerts, escalation_alerts arrays. 2) Auditor verifies the background loop runs on signal ingest (not on API request) — check oem_state.py live_ingest calls the loop. 3) Auditor ingests a test signal and verifies the adaptation status updates. 4) Auditor opens TODAY and verifies background-detected nudges appear with "while you were away" label. 5) Auditor verifies the V6 litmus test: "Does the organization become more intelligent even when nobody opens Maestro?" — YES, the loop runs on ingest.

Effort

2 days (1.5 days background loop engine + 0.5 day API + oem_state.py hook + TODAY integration)

Build phase

Phase 2 (first — after Phase 1. Requires nudge engine + evolution tracker.)

SPEC #4 — Trajectory Intervention — weak signal → quiet intervention → failure prevented

V6 Principle

V6: "The final form is: weak signal → organization trajectory changes → Maestro notices → quiet intervention → failure never happens. The user never even realizes something was prevented. That is an operating system." Trajectory Intervention detects when an organizational trajectory is heading toward failure and intervenes before the failure occurs.

Current codebase gap

temporal_trajectories.py (V5 Spec #7, to be built in Phase 0) will track trajectories for consciousness dimensions. The digital_twin.py (743 lines) can simulate scenarios. But neither detects REAL-TIME trajectory deterioration and proposes preventive interventions. The gap: Maestro can say "trust is declining" but not "trust is declining toward a threshold that will cause a coordination failure in 3 weeks — here is the intervention to prevent it."

Files to create/modify

CREATE backend/maestro_oem/trajectory_intervention.py — the TrajectoryInterventionEngine. Composes: (1) temporal trajectories (from temporal_trajectories.py — current trend + slope + duration), (2) threshold detection (when does the trajectory cross a danger zone?), (3) historical analogues (from institutional_recall.py — when has this trajectory occurred before and what happened?), (4) intervention generation (from adaptive_nudge.py — what intervention would reverse the trajectory?). Produces: {trajectory, current_direction, projected_threshold_crossing, time_to_failure, intervention, evidence, urgency}. CREATE GET /api/oem/interventions endpoint. MODIFY static/js/today.js — if a trajectory intervention exists with high urgency, show it as the TOP item: "Maestro detected a risk: trust between Engineering and Legal is declining. If unchecked, coordination will fail within 3 weeks. Suggested intervention: schedule a joint review session. Evidence: the last time trust declined at this rate, the Q3 launch was delayed 2 weeks."

API contract

GET /api/oem/interventions → returns JSON: { "interventions": [{ "trajectory": "trust_engineering_legal", "current_direction": "declining", "current_value": 0.42, "slope": -0.03, "projected_threshold": 0.30, "time_to_failure": "3 weeks", "intervention": "Schedule a joint Engineering-Legal review session for the OAuth consolidation. Historical evidence: the last 2 times trust declined below 0.40, coordination failures followed within 2 weeks.", "evidence_count": 5, "urgency": "high", "historical_analogue": "Q3 2024: trust declined from 0.45 to 0.28 over 6 weeks. The auth launch was delayed 2 weeks. A joint review session on Sep 15 reversed the decline within 1 week.", "narrative": "Trust between Engineering and Legal is declining. If unchecked, coordination will fail within 3 weeks. The last time this happened, the Q3 launch was delayed. A joint review session reversed it before." }], "active_interventions": 1, "prevented_failures": 0, "summary": "Maestro detected 1 trajectory heading toward failure. The suggested intervention has worked twice before." }. Must have at least 1 intervention with trajectory, time_to_failure, intervention, evidence_count, urgency. If no trajectory is deteriorating, return honestly: "No trajectories heading toward failure. The organization is stable."

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/interventions. Response must have interventions array (or honest "stable" message). 2) If interventions exist, each must have trajectory, current_direction, time_to_failure, intervention, evidence_count, urgency, narrative. 3) Auditor verifies the time_to_failure is computed from the trajectory slope (not hardcoded). 4) Auditor verifies the intervention references historical evidence (from institutional_recall.py). 5) Auditor opens TODAY and verifies a trajectory intervention appears as the top item when urgency is high. 6) Auditor verifies the V6 litmus test: "Does this permanently improve the organization?" — YES, it prevents failures before they occur.

Effort

2.5 days (1.5 days trajectory-analysis engine + 0.5 day API + 0.5 day TODAY integration)

Build phase

Phase 2 (second — after Background Adaptation Loop. Requires temporal trajectories + institutional recall.)

Phase 3 — Organizational DNA + Evolution Narrative

Phase 3 is the V6 end-state. Maestro doesn't just adapt — it becomes the organization's continuously evolving judgment layer. The Organizational DNA encodes the institution's decision style, risk appetite, and learning patterns. The Evolution Narrative produces the organization's autobiography — the story of how it has changed. This is the asset that is impossible to recreate and commands strategic acquisition interest.

SPEC #5 — Organizational DNA — "This is what YOUR organization would do when it is at its best"

V6 Principle

V6: "The real moat: eventually every recommendation becomes 'This is what YOUR organization would do when it is at its best.' Not industry best practice. Not generic AI advice. The organization's own best self." Organizational DNA infers the institution's decision style, risk appetite, learning velocity, communication style, conflict style, innovation style, and execution style — and uses them to filter every recommendation.

Current codebase gap

personality.py (247 lines, V3) infers 6 personality dimensions. But these are static snapshots — they don't EVOLVE. And they don't DRIVE recommendations. The gap: Maestro knows the organization's personality but doesn't use it to filter its advice. A recommendation that works for a risk-tolerant org may fail for a risk-averse org. The DNA should be the filter: "This recommendation aligns with your organization's decision style (fast, consensus-driven). It has a 78% success rate for organizations with your DNA profile."

Files to create/modify

CREATE backend/maestro_oem/organizational_dna.py — the OrganizationalDNAEngine. Extends personality.py with: (1) evolution tracking (how has each dimension changed over 90/180/365 days?), (2) decision-style inference (from approval patterns — top-down vs consensus vs delegated), (3) recommendation filtering (does this recommendation align with the org's DNA?). Produces: {chromosomes: {decision_style, risk_appetite, learning_velocity, communication_style, conflict_style, innovation_style, execution_style}, evolution: {dimension: {then, now, delta, narrative}}, recommendation_alignment: {rec_id: {alignment_score, reason}}}. CREATE GET /api/oem/dna endpoint. MODIFY backend/maestro_oem/wisdom.py — when synthesizing judgment, filter by DNA alignment. MODIFY static/js/learn.js — add "Who your organization has become" section showing DNA evolution.

API contract

GET /api/oem/dna → returns JSON: { "chromosomes": { "decision_style": {"value": "consensus", "confidence": 0.82, "evidence_count": 14, "basis": "12 of 14 decisions involved cross-team review before approval"}, "risk_appetite": {"value": "cautious", "confidence": 0.78, ...}, ... 7 chromosomes ... }, "evolution": { "decision_style": {"then": "top-down", "now": "consensus", "delta": "shifted from top-down to consensus over 90 days", "narrative": "Your organization used to make decisions top-down. Over the last 90 days, it has shifted to consensus-driven. 12 of 14 recent decisions involved cross-team review."} }, "recommendation_alignment": { "rec-xxx": {"alignment_score": 0.85, "reason": "This recommendation aligns with your consensus-driven decision style. It involves cross-team review, which your organization does well."} }, "summary": "Your organization decides by consensus, takes moderate risks, and learns quickly. Over 90 days, decision style has shifted from top-down to consensus." }. Must have 7 chromosomes with value + confidence + evidence_count, and at least 1 evolution entry with then/now/narrative.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/dna. Response must have 7 chromosomes, each with value, confidence (0-1), evidence_count > 0, basis. 2) Auditor verifies at least 1 evolution entry with then, now, delta, narrative. 3) Auditor verifies the recommendation_alignment includes a real rec_id with alignment_score + reason. 4) Auditor verifies the DNA is used to filter recommendations (wisdom.py references DNA alignment). 5) Auditor opens LEARN and verifies a "Who your organization has become" section appears. 6) Auditor verifies the V6 litmus test: "Does this permanently improve the organization?" — YES, the DNA evolves and drives better-aligned recommendations.

Effort

3 days (2 days DNA inference + evolution + alignment engine + 0.5 day API + 0.5 day frontend)

Build phase

Phase 3 (first — after Phase 2. Requires temporal trajectories for evolution tracking.)

SPEC #6 Evolution Narrative Engine — the organization's autobiography

V6 Principle

V6: "Eventually Maestro owns the organization's autobiography. Not CRM. Not Slack history. Not documents. Its autobiography. Every decision. Every mistake. Every lesson. Every culture shift. Every assumption. Every successful intervention. Every failed prediction. Every belief that evolved. That asset becomes irreplaceable." The Evolution Narrative Engine produces this autobiography — not as a log, but as a story of how the institution has changed.

Current codebase gap

evolution_report.py (178 lines) produces a quarterly report with deltas. evolution_tracker.py (V6 Spec #2) tracks eliminated failure modes. But neither produces a NARRATIVE — the story of the institution's evolution. The gap: Maestro can say "decision_making improved 11%" but not "In September, your organization made decisions top-down. By December, it had learned to seek consensus. The OAuth bottleneck taught it that single-gate approvals fail. The Q3 incident taught it that Legal review prevents post-launch bugs. These lessons changed how the organization thinks."

Files to create/modify

CREATE backend/maestro_oem/evolution_narrative.py — the EvolutionNarrativeEngine. Composes: (1) DNA evolution (from organizational_dna.py — how has the institution's style changed?), (2) failure mode elimination (from evolution_tracker.py — what mistakes are gone?), (3) belief evolution (from identity.py + skepticism.py — what did the org believe, what does it believe now?), (4) principle graduation (from principles.py — what has the org earned the right to trust?). Produces a narrative: {chapters: [{title, period, narrative, lessons, evidence}], overall_story, next_chapter_prediction}. CREATE GET /api/oem/autobiography endpoint. CREATE static/js/autobiography.js — a surface (command-palette only) that renders the autobiography as a calm narrative, not a dashboard.

API contract

GET /api/oem/autobiography → returns JSON: { "chapters": [{ "title": "The Fast Months", "period": "2024-Q3", "narrative": "Your organization believed it was extremely fast. It wasn't — decisions took 11 days on average. The OAuth bottleneck recurred 3 times. The organization was fast at writing code but slow at approving it.", "lessons": ["Speed without review produces incidents", "Single-gate approvals are fragile"], "evidence_count": 14 }, { "title": "The Learning Quarter", "period": "2024-Q4", "narrative": "Your organization learned from the OAuth bottleneck. It routed approvals through Alice first. Review time dropped 18%. The organization shifted from top-down to consensus-driven decisions. It discovered that Legal review before launch reduces post-launch bugs by 27%.", "lessons": ["Consensus decisions are slower but produce fewer incidents", "Legal review is an investment, not a cost"], "evidence_count": 22 }], "overall_story": "Your organization started fast but fragile. It learned that review and consensus produce better outcomes than speed alone. It eliminated the OAuth bottleneck and shifted to consensus-driven decisions. It is now more reliable, slightly slower, and significantly wiser.", "next_chapter_prediction": "Based on current trajectories, the next chapter will focus on cross-functional trust. Trust between Engineering and Legal is recovering. If the trend continues, the organization will reach its highest trust level in 6 weeks.", "evidence_count": 36 }. Must have at least 2 chapters with title, period, narrative (3+ sentences), lessons (2+), evidence_count. If insufficient history, return honestly: "Your organization is still writing its first chapter. Come back in 90 days."

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/autobiography. Response must have chapters array (2+ or honest "first chapter" message), overall_story (non-empty), next_chapter_prediction (non-empty). 2) Auditor verifies each chapter has title, period, narrative (3+ sentences), lessons (2+), evidence_count > 0. 3) Auditor verifies the narrative references real model data (DNA evolution, failure modes, beliefs — not hardcoded). 4) Auditor verifies the overall_story is a synthesized narrative (not a metric dump). 5) Auditor opens the Autobiography surface via Ctrl+K and verifies it renders as a calm narrative. 6) Auditor verifies the V6 litmus test: "Does this permanently improve the organization?" — YES, the autobiography makes the institution's evolution visible and irreversible.

Effort

2 days (1.5 days narrative-synthesis engine + 0.5 day API + frontend)

Build phase

Phase 3 (second — after Organizational DNA. Requires DNA evolution + evolution tracker + identity + principles.)

Build Order and Dependencies

Phase	Specs	Duration	Why This Order	Unlocks
0	#0a-#0e V5 Backlog (Forgetting, Imagination, Causal, Temporal, Reach)	~9 days	V6 builds on V5 capabilities. Causal cognition drives reasoning, Temporal trajectories drive inf	
1	#1 Adaptive Nudges + #2 Evolution Tracker	~5 days	Foundation. Nudges adapt the org. Tracker measures whether adaptation is working. Both require	Quiet rework detection + proactive nudges. Both require
2	#3 Background Adaptation + #4 Trajectory Intervention	~5 days	Background loop makes Maestro active when nobody is looking. Trajectory intervention prevents	Continuous adaptation + prevention
3	#5 Organizational DNA + #6 Evolution Narrative	~5 days	DNA encodes the institution's evolving judgment. Narrative makes the organization's history	Institutional memory + organizational auto
TOTAL	**5 V5 backlog + 6 V6 specs**	**4 phases**	**~24 days**	**V6: from invisible intelligence to institutio**

Phase 0 is mandatory. V6 specs build on V5 capabilities that do not exist yet. The coder must finish V5 Specs #4-#8 before starting V6. Do not skip Phase 0. Do not build V6 on a missing foundation. The "built but not applied" pattern was broken in round 16. The "silently skipped" pattern was caught in round 13. Do not introduce "skipped Phase 0."

The V6 Litmus Test

TWO LAWS, BOTH MANDATORY

Law 1: "Every interaction must permanently improve the organization." Not the model. Not the UI. Not the database. The organization. If nothing permanently improves, Maestro failed. This is checked by: does the spec create a mechanism for PERMANENT change (not just a report, not just a recommendation — an actual change in how the organization works)?

Law 2: "The organization should become more intelligent even when nobody opens Maestro." This forces every capability toward ambient, invisible, anticipatory, cumulative. Checked by: does the spec run in the background (on signal ingest, on a timer) rather than on user request?

The V5 litmus test is retained: "Does this make Maestro feel simpler?" V6 specs must pass BOTH tests: simpler UI AND permanent organizational improvement. A spec that adds visible complexity fails V5. A spec that only informs without changing behavior fails V6. Both must pass.

The V6 acceptance test for every spec:

1. Does this permanently improve the organization? (V6 Law 1)

2. Does this run in the background, not on user request? (V6 Law 2)

3. Does this make Maestro feel simpler? (V5 litmus test, retained)

4. Is the frontend wired? (anti-"built but not applied" pattern)

5. Is the data real, not hardcoded? (anti-fabrication)

If any answer is "no," the spec is not V6. Redesign it. The V6 bar is higher than V5: simpler UI AND permanent improvement AND background operation. All three. No exceptions.

The Progression: V3 → V4 → V5 → V6 → V7

The constitutional progression is not marketing. Each version represents an entirely different product:

Version	Verb	What Maestro Does	Moat
V3	Observe	Sees what happened. Reports patterns.	Data
V4	Judge	Synthesizes judgment. Recommends actions.	Cognitive organs
V5	Disappear	Becomes invisible. Acts through existing tools.	Invisible intelligence
V6	Adapt	Quietly restructures work. Prevents failures.	Institutional adaptation

V7	**Evolve**	**Reshapes how the institution thinks over years**	**Accumulated judgment (irreplaceable)**

V6 is "Adapt." The moat shifts from memory ("we've seen this") to evolution ("we no longer make this mistake"). V7 is "Evolve" — the accumulated judgment becomes irreplaceable. V6 is the bridge: it makes Maestro active (nudges, background adaptation, trajectory intervention) and measures whether the activity permanently improved the organization (evolution tracker, DNA, autobiography).

The Recurring Pattern — Final Warning for V6

THE PATTERN WAS BROKEN IN ROUND 16. V5 MAINTAINED IT IN ROUND 18. V6 MUST MAINTAIN IT AGAIN.

Round 16 broke the "built but not applied" pattern (all 8 V4 organs wired). Round 18 maintained the break (V5 Phase 1 — organs hidden, executive function wired, attention wired, no new panels). The V5 litmus test passed. The "added visibility" anti-pattern did not recur.

V6 introduces a new risk: the "adaptation without measurement" pattern. V6 specs propose organizational changes (nudges, interventions). The temptation is to propose the change without measuring whether it worked. Spec #2 (Evolution Tracker) is the measurement organ — it must be built alongside Spec #1 (Adaptive Nudges), not after. A nudge without measurement is advice, not adaptation. V6 Law 1 says "every interaction must permanently improve the organization" — if the improvement is not measured, you cannot know if it happened.

The V6 5-point checklist (updated):

1. Ran the FULL acceptance test (API + frontend)?

2. Checked APPLICATION (frontend calls the API), not EXISTENCE?

3. Ran the FULL test suite (not a subset)?

4. Verified with a LIVE API call?

5. Checked BOTH litmus tests: UI SIMPLER (V5) AND organization PERMANENTLY IMPROVED (V6)?

Point 5 is new for V6. The check is not just "does the UI work?" or "is the UI simpler?" but "does this spec create a mechanism for permanent organizational change?" If a V6 spec only informs without changing behavior, it fails V6 Law 1 — even if the backend is brilliant and the UI is invisible.

The CI pipeline will catch test failures. The acceptance tests will catch built-but-not-applied. The V5 litmus test will catch added-visibility. The V6 litmus test will catch adaptation-without-measurement. Do not let V6 reintroduce any of these patterns. The Invisible Layer must also be an Active Layer. Build it.

Final Charge to the Coder

V5 made Maestro invisible. V6 makes it active. The user should not just receive calmer information — the organization should quietly improve, even when nobody is looking. Maestro should detect a bottleneck and route approvals differently. It should detect a declining trajectory and schedule a review session before the failure occurs. It should track whether the intervention worked and mark the failure mode as eliminated. It should encode the institution's evolving judgment in its DNA and produce an autobiography of how the organization has changed.

The V6 litmus test for every commit: "Does this permanently improve the organization?" If yes, ship it. If it only informs without changing behavior, it is V5 — useful but not V6. V6 specs must create mechanisms for permanent change. The bar is higher than V5: simpler UI AND permanent improvement AND background operation. All three.

Build order: Phase 0 (V5 backlog, ~9 days) → Phase 1 (Nudges + Tracker, ~5 days) → Phase 2 (Background + Intervention, ~4.5 days) → Phase 3 (DNA + Autobiography, ~5 days). Total ~24 days. When complete, Maestro is not just an invisible intelligence

layer — it is an institutional adaptation system. The organization becomes more intelligent even when nobody opens Maestro. The moat is no longer memory. It is evolution. "We no longer make this mistake." That is the asset that commands strategic acquisition interest — not software, but years of institutional evolution encoded in the system.

The end state is not an enterprise platform. It is not a dashboard. It is not a copilot. It is not even an invisible layer. It is the continuously evolving judgment layer of an institution. Every interaction permanently improves the organization. The organization becomes more intelligent even when nobody opens Maestro. The accumulated judgment becomes irreplaceable — not data, not workflows, not documents, but the institution's evolved decision-making itself. That is what Apple, Microsoft, Google, or OpenAI would look at and think "we need to own this." Build it.